

Project 5: Short-Term Process Scheduler

Nolan Tuttle & Mathew Hobson

College of Engineering and Technology, Grand Canyon University

CST-315: Operating Systems Lecture and Lab

Professor Citro

April 6, 2025

Summary

This program is an improvement of our previous command-line interpreter program created in Project 3. Our goal with this improved version was to implement a Round-Robin (RR) process scheduler that allocates a time quantum to all running process, preempting each one and moving on to the next after the time quantum has passed until all processes in the queue are completed. We removed the batch file functionality but kept all core features like tokenized inputs, allowing for command separation with semicolons. Each time a command is given by the user to the interpreter, it will add it to the queue and run all processes in the queue, displaying an updated process table for the user to see each time the dispatcher switches processes. An important design choice we made was creating a data structure for each process, that way each process can keep track of its own command (like 'cd' or 'ls'), whether it is finished or not, its process ID, and more. The process table shows all processes running, their id, the name, and the state they are in, which could be running, ready, or blocked. We also added support for the 'procs' command, allowing for the -a and '-a -si' options to be specified depending on how the user wants the output sorted. This command will show detailed information about all processes running on the machine, sorted by their id, alphabetically, or unsorted. The program also displays the average turnaround time for each process once they are all finished, allowing for turnaround time analysis and selection of the best time quantum for the situation.

It is important to note that we added 3 demonstration processes that run idly: "sleep 2", "echo idle1 > /dev/null", and "sleep 4". When a command is entered, it will be put at the end of the queue with these commands in front, showing the process table each time a context switch is performed. Also, the 'procs' command will run and then exit the program when used.

Code Functionality

Our code consists of 3 key parts: displaying the process states, queueing up a new process, and the RR scheduling algorithm.

1. Displaying Process States

- a. For this, we simply abstracted out a print method that will show a process ID, name, and status given its index in the process queue.

```
// Displays currently active processes and their states
void displayProcessStates(int currentIndex)
{
    printf("\n+-----+-----+-----+\n");
    printf("| ID | Command | Status | \n");
    printf("+-----+-----+-----+\n");
    for (int i = 0; i < processCount; i++)
    {
        printf("| %2d | %-26s | ", i, processQueue[i].command);
        if (processQueue[i].finished)
        {
            printf("FINISHED | \n");
        }
        else if (i == currentIndex)
        {
            printf("RUNNING | \n");
        }
        else
        {
            printf("BLOCKED | \n");
        }
    }
    printf("+-----+-----+-----+\n\n");
}
```

Method for printing out the process states

2. Queueing a New Process

- a. We abstracted out a method for adding a process to the process queue, simply taking a pointer to an array of characters and updating each of the variables of our Process data structures within the process queue including the actual command string, the finished status, process ID, etc.

```

void enqueueCommand(const char *cmd)
{
    if (processCount >= 10)
        return;
    strncpy(processQueue[processCount].command, cmd, sizeof(processQueue[processCount].command));
    processQueue[processCount].finished = 0; // Sets finished status to 0 (Unfinished process)
    processQueue[processCount].pid = -1;     // Sets default process ID to -1
    processCount++;
    processQueue[processCount].arrivalTime = clock(); // Store the arrival time
    processQueue[processCount].completionTime = 0;   // Sets completion time to 0 as default
}

```

Method for queueing a given command to the process queue.

3. Scheduling Algorithm

- a. We modified our forking process to create child processes, then immediately pause them and run them in the order of the queue, giving each process a time quantum to use when running.

```

for (int i = 0; i < processCount; i++) // This loop queues up all processes and pauses them immediately to be run by the scheduler
{
    if (strncmp(processQueue[i].command, "cd ", 3) == 0)
    {
        sleep(time_quantum); // Simulate time quantum used

        // Extract the path and change directory
        char *path = strtok(processQueue[i].command, " ");
        path = strtok(NULL, " "); // The argument for cd
        if (path)
            chdir(path);

        processQueue[i].finished = 1;
        finishedProcesses++;
        displayProcessStates(i);
        continue;
    }

    pid_t pid = fork(); // Forks a new process

    if (pid == 0) // Child process
    {
        char *args[] = {"./bin/bash", "-c", processQueue[i].command, NULL};
        if (execv("./bin/bash", args) == -1)
        {
            perror("exec failed");
            exit(1);
        }
    }
    else if (pid > 0) // Parent process
    {
        processQueue[i].pid = pid;
        kill(pid, SIGSTOP);
    }
}

```

Forking child processes and pausing using kill() as well as logic for changing directory

```

while (finishedProcesses < processCount) // Scheduler loop that runs processes waiting in the queue
{
    for (int i = 0; i < processCount; i++)
    {
        if (processQueue[i].finished)
            continue;

        kill(processQueue[i].pid, SIGCONT); // Signals current process to start running
        displayProcessStates(i);
        sleep(time_quantum);                // Runs only for time quantum
        kill(processQueue[i].pid, SIGSTOP); // Pauses the process

        int status;
        pid_t result = waitpid(processQueue[i].pid, &status, WNOHANG); // Checks if the current process is finished
        if (result == processQueue[i].pid)                               // If it has finished, sets condition and increments finished counter
        {
            processQueue[i].finished = 1;
            processQueue[i].completionTime = clock(); // Store the completion time when process finishes
            finishedProcesses++;
        }
    }
}

```

Loop for executing queued up processes, printing the process states and sleeping for the time quantum each iteration until all processes finish

Code Execution

```
seaShell$ procs
PID USER      STAT  COMM
488  _accessoryupdater S    /usr/sbin/distnoted
198  _analyticsd   S    /System/Library/PrivateFrameworks/CoreAnalytics.framework/Support/analyticsd
485  _appleevents  S    /usr/sbin/distnoted
316  _appleevents  S    /System/Library/CoreServices/appleeventsd
692  _applepay     S    /usr/sbin/distnoted
78632 _applepay     S    /usr/libexec/nfcd
421  _assetcache   S    /usr/sbin/distnoted
78380 _assetcache   S    /System/Library/PrivateFrameworks/AssetCacheServices.framework/Versions/A/XPCServices/AssetCacheLocatorService.xpc/Contents/MacOS/AssetCacheLocatorService
78431 _assetcache   S    /usr/libexec/AssetCache/AssetCache
32456 _audiomxd     S    /usr/sbin/distnoted
77109 _audiomxd     S    /usr/libexec/audiomxd
77085 _biome        S    /System/Library/PrivateFrameworks/BiomeStreams.framework/Support/biomed
276  _cmiodassistant S    /usr/sbin/distnoted
319  _cmiodassistant S    /System/Library/Frameworks/CoreMediaIO.framework/Versions/A/Resources/UVCAssistant.systemextension/Contents/MacOS/UVCAssistant
314  _cmiodassistant S    /usr/libexec/cameracaptured
313  _cmiodassistant S    /usr/sbin/applehi3camerad
242  _coreaudioid  S    /usr/sbin/distnoted
210  _coreaudioid  S    /usr/sbin/coreaudioid
317  _coreaudioid  S    Core Audio Driver (BlackHole16ch.driver)
359  _coreaudioid  S    Core Audio Driver (MSTeamsAudioDevice.driver)
387  _ctkd         S    /System/Library/Frameworks/CryptoTokenKit.framework/ctkd
11362 _diskimagesiod S    /usr/libexec/diskimagesiod
```

Running the 'procs' command

```
seaShell$ procs -a
1      0 root      31.0  0.2 411054448 15216 ??    Ss   2Mar25 38:26.09 /sbin/launchd
1406   1 nolan     2.7   0.4 412040576 36272 ??    S    2Mar25 2:21.95 /System/Applications/Utilities/Terminal.app/Contents/MacOS/Terminal
78944 78936 nolan  2.6   0.0 410724592 1760 ttys000 S+   6:53PM 0:00.01 sh -c ps -eo pid,ppid,user,%cpu,%mem,vsz,rss,stat,stime,time,command|sort -k4 -nr
78463 1 nolan     2.2   3.8 419101008 322528 ??    S    Fri04PM 17:56.45 /Applications/Microsoft Word.app/Contents/MacOS/Microsoft Word
76240 76225 nolan  1.9   1.3 1627174448 107680 ??    S    3:48PM 5:06.93 /Applications/Discord.app/Contents/Frameworks/Discord Helper (Renderer).app/Contents/MacOS/Discord Helper (Renderer) --type=renderer --user-data-dir=/Users/nolan/Library/Application Support/discord --secure-schemes=disclip,sentry-ipc --bypasscsp-schemes=sentry-ipc --cors-schemes=sentry-ipc --fetch-schemes=disclip,sentry-ipc --app-path=/Applications/Discord.app/Contents/Resources/app.asar --no-sandbox --no-zygote --enable-blink-features=EnumerateDevices,AudioOutputDevices --autoplay --policy=no-user-gesture-required --lang=en-US --num-raster-threads=4 --enable-zero-copy --enable-gpu-memory-buffer-compositor-resources --enable-main-frame-before-activation --renderer-client-id=6 --time-ticks-at-unix-epoch=-1743718216920213 --launch-time-ticks=261512239108 --shared-file-s --field-trial-handle=1718379636,r,8903796892078359399,15911293792559005852,262144 --disable-features=AllowAggressiveThrottlingWithWebsocket,HardwareMediaKeyHandling,IntensiveWakeUpThrottling,MacWebContentsOcclusion,MediaSessionService,ScreenCaptureKitMac,ScreenCaptureKitMacScreen,ScreenCaptureKitMacWindow,ScreenCaptureKitPickerScreen,ScreenCaptureKitStreamPickerSonoma,SpareRendererForSitePerProcess,TimeoutHangingVideoCaptureStreams,UseEcoQoSForBackgroundProcess,UseSCContentSharingPicker,WarmScreenCaptureSonoma,WinRetrieveSuggestionsOnlyOnDemand --variations-seed-version --enable-node-leakage-in-renderers
158    1 windowserver 0.9   1.0 413163648 88048 ??    Ss   2Mar25 388:04.25 /System/Library/PrivateFrameworks/SkyLight.framework/Resources/WindowServer -daemon
78836 54560 nolan  0.6   0.0 410592912 688 ttys000 S+   6:48PM 0:00.01 ./seaShell
210    1 _coreaudioid 0.4   0.2 410430800 17968 ??    Ss   2Mar25 78:46.18 /usr/sbin/coreaudioid
104    1 root      0.2   0.1 410365776 7840 ??    Ss   2Mar25 8:21.45 /System/Library/CoreServices/powerd.bundle/powerd
78946 78944 nolan  0.1   0.0 410724032 1328 ttys000 R+   6:53PM 0:00.00 sort -k4 -nr
78945 78944 root   0.1   0.0 410725312 2224 ttys000 R+   6:53PM 0:00.00 ps -eo pid,ppid,user,%cpu,%mem,vsz,rss,stat,stime,time,command
```

Running the 'procs' command with option '-a' specified

```
(base) Nolan-MacBook-Air:Project 5 Short-Term Process Scheduler nolan$ ./seaShell
<CTRL+X to end execution>
<CTRL+B to exit the shell>
seaShell$ procs -a -si
PID PPID USER      %CPU %MEM    VSZ   RSS TTY      STAT STARTED     TIME COMMAND
1      0 root      0.1  0.2 411053888 12592 ??    Ss   2Mar25 37:38.59 /sbin/launchd
90     1 root      0.0  0.2 410441904 18976 ??    Ss   2Mar25 51:19.80 /usr/libexec/nfcd
92     1 root      0.0  0.1 410365792 7328 ??    Ss   2Mar25 2:58.82 /usr/libexec/audiomxd
94     1 root      0.0  0.0 410164752 1872 ??    Ss   2Mar25 0:08.95 /System/Library/PrivateFrameworks/AssetCacheServices.framework/Versions/A/XPCServices/AssetCacheLocatorService.xpc/Contents/MacOS/AssetCacheLocatorService
95     1 root      0.0  0.1 410481024 5952 ??    Ss   2Mar25 89:11.86 /System/Library/PrivateFrameworks/AssetCacheServices.framework/Versions/A/XPCServices/AssetCacheLocatorService.xpc/Contents/MacOS/AssetCacheLocatorService
96     1 root      0.0  0.1 410367440 5392 ??    Ss   2Mar25 0:31.56 /System/Library/PrivateFrameworks/AssetCacheServices.framework/Versions/A/XPCServices/AssetCacheLocatorService.xpc/Contents/MacOS/AssetCacheLocatorService
99     1 root      0.0  0.1 410378768 9632 ??    Ss   2Mar25 1:29.39 /usr/sbin/coreaudioid
102    1 root      0.0  0.1 410367888 6192 ??    Ss   2Mar25 10:40.16 /usr/libexec/AssetCache/AssetCache
104    1 root      0.0  0.1 410365776 7712 ??    Ss   2Mar25 8:12.59 /System/Library/PrivateFrameworks/AssetCacheServices.framework/Versions/A/XPCServices/AssetCacheLocatorService.xpc/Contents/MacOS/AssetCacheLocatorService
105    1 root      0.0  0.0 410294368 1984 ??    Ss   2Mar25 0:00.05 /usr/libexec/audiomxd
106    1 _biome     0.0  0.1 410363744 7232 ??    Ss   2Mar25 0:29.77 /System/Library/PrivateFrameworks/BiomeStreams.framework/Support/biomed
110    1 root      0.0  0.0 410363344 1952 ??    Ss   2Mar25 0:00.65 /usr/libexec/audiomxd
115    1 root      0.0  0.0 410164336 2208 ??    Ss   2Mar25 0:07.08 /usr/libexec/audiomxd
119    1 root      0.0  0.3 410715808 23248 ??    Ss   2Mar25 30:51.55 /System/Library/PrivateFrameworks/AssetCacheServices.framework/Versions/A/XPCServices/AssetCacheLocatorService.xpc/Contents/MacOS/AssetCacheLocatorService
```

Running the 'procs' command with options '-a' and '-si' specified

```
((base) Nolan's-MacBook-Air:Project 5 Short-Term Scheduler nolan$ ./seaShell
<CTRL+X to end execution>
<CTRL+B to exit the shell>
seaShell$ ls
```

ID	Command	Status
0	sleep 2	RUNNING
1	echo idle1 > /dev/null	BLOCKED
2	sleep 4	BLOCKED
3	ls	BLOCKED

ID	Command	Status
0	sleep 2	BLOCKED
1	echo idle1 > /dev/null	RUNNING
2	sleep 4	BLOCKED
3	ls	BLOCKED

ID	Command	Status
0	sleep 2	BLOCKED
1	echo idle1 > /dev/null	FINISHED
2	sleep 4	RUNNING
3	ls	BLOCKED

ID	Command	Status
0	sleep 2	BLOCKED
1	echo idle1 > /dev/null	FINISHED
2	sleep 4	BLOCKED
3	ls	RUNNING

README.txt seaShell seaShell.c test

ID	Command	Status
0	sleep 2	RUNNING
1	echo idle1 > /dev/null	FINISHED
2	sleep 4	BLOCKED
3	ls	FINISHED

ID	Command	Status
0	sleep 2	FINISHED
1	echo idle1 > /dev/null	FINISHED
2	sleep 4	RUNNING
3	ls	FINISHED

```
seaShell$
```

Running the 'ls' command. Notice how it only runs when the dispatcher reaches it in the queue, every process before must receive its time quantum first.

Scheduling Algorithm Choice

The Round-Robin scheduling approach is great for time-sharing systems and offers some improvements over other scheduling algorithms. This is because unlike First-Come First-Served (FCFS) scheduling, if for some reason a very lengthy process ends up first in the queue, it will not slow down the other processes since the dispatcher will switch after the time quantum has expired. For Shortest process next (SPN) scheduling, this is not an issue, however it may prove difficult to implement as it requires processes to declare how much time they will need before entering the queue or for an estimate to be available. Also, there is a risk of starvation of longer processes with SPN, if the shortest jobs being queued up complete and reenter the queue before the longer processes can be queued up, the longer processes will not be serviced and remain in the queue for a long time. However, a big disadvantage of Round-Robin scheduling is the overhead it generates, since it must generate an interrupt at periodic intervals, usually dependent on the clock (Stallings, 2019). Another possible disadvantage is poor service for I/O bound processes. Since I/O bound processes tend to have a shorter processor burst, which is the time between I/O executions, than processor-bound processes, they might not receive a fair portion of the time quantum (Stallings, 2019). If a processor-bound process gets its time quantum, executes using the full quantum and re-enters the queue whereas an I/O bound one will be blocked because of I/O operations before entering the queue again, the processor-bound process will be able to exit and enter the queue much more often, receiving much more of the time quantum.

Parameter Adjustment

We implemented an adjustable time quantum, allowing use of a system call to change the quantum of the short-term scheduler. Interesting effects occur when adjusting this parameter, for example when the time quantum is large enough that all processes are completed in 1 time quantum or less, it degrades into FCFS (Stallings, 2019). However, if the time quantum is extremely small, this will result in many more context switches for the dispatcher to perform, drastically increasing overhead. The optimal time quantum is large with respect to the context switch time, for example if the context switch time is 10 percent of the time quantum, approximately 10 percent of the CPU time will be spent on context switches (Stallings, 2019). As a rule of thumb, 80 percent of CPU bursts should be shorter than the time quantum (Stallings 2019). Measuring the effectiveness of a Round-Robin scheduler can be done using turnaround time, which is the amount of time to execute a particular process. For example, using our 'setquantum' command, when the quantum is set to 1 second instead of the default 2 seconds, the idle process with ID 0 receives much slower service because it does not complete during the first round. The overhead is also much worse because instead of 4 context switches, there are 6 in this situation.

```
seaShell$ setquantum 1
Time quantum set to 1 seconds.
```

ID	Command	Status
0	sleep 2	RUNNING
1	echo idle1 > /dev/null	BLOCKED
2	sleep 4	BLOCKED

ID	Command	Status
0	sleep 2	BLOCKED
1	echo idle1 > /dev/null	RUNNING
2	sleep 4	BLOCKED

ID	Command	Status
0	sleep 2	BLOCKED
1	echo idle1 > /dev/null	FINISHED
2	sleep 4	RUNNING

ID	Command	Status
0	sleep 2	RUNNING
1	echo idle1 > /dev/null	FINISHED
2	sleep 4	BLOCKED

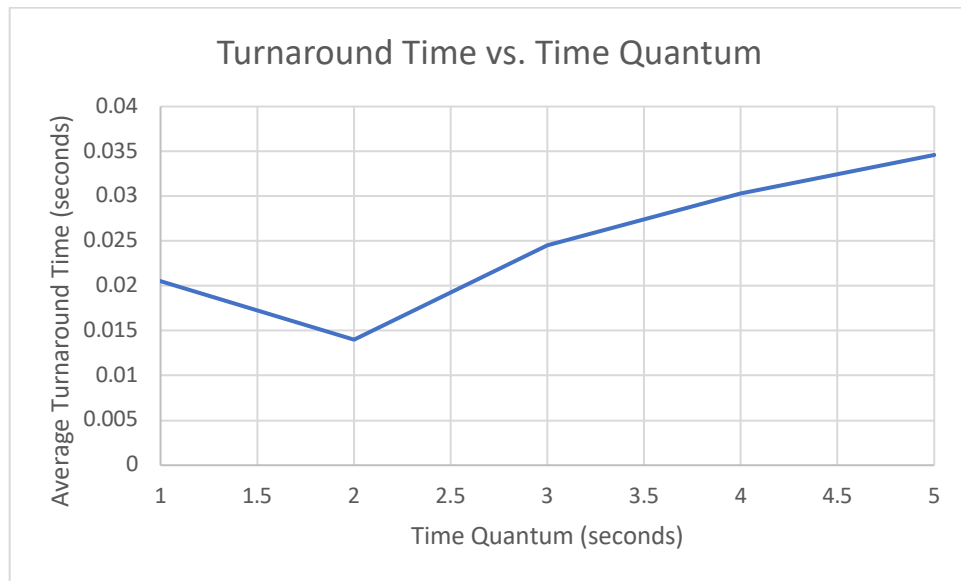
ID	Command	Status
0	sleep 2	FINISHED
1	echo idle1 > /dev/null	FINISHED
2	sleep 4	RUNNING

ID	Command	Status
0	sleep 2	FINISHED
1	echo idle1 > /dev/null	FINISHED
2	sleep 4	RUNNING

Setting the time quantum to 1 has the effect of increasing context switches, increasing overhead.

Data Observations

To further observe the effects that the time quantum has on our scheduler, we plotted the average turnaround time for each process when executing CPU bursts for four processes, 'sleep 2', 'echo idle1 > /dev/null', 'sleep 4', and 'ls'. We tested this for time quantum ranging 1-5 and created the following graph:



This graph demonstrates that the most effective quantum (lowest average turnaround time) turned out to be 2 seconds in this scenario. Obviously, this depends heavily on the different processes doing CPU bursts because processes vary a lot in time required for execution, but with this measurement it is much clearer which time quantum should be used.

References

Stallings, W. (2019). *Operating Systems: Internals and Design Principles* (9th ed.). Pearson.

Tuttle, N., Hobson, M., (2025) *CST315* GitHub. <https://github.com/nolantuttle/CST315.git>